CREDIT CARD FRAUD DETECTION

- Stucchi Jacopo 47294A
- Riccardo Cambianica 53147A

New Generation Data Models and DBMSs
Academic year: 2024/2025



CREATION OF A NOSQL DATABASE FOR CREDIT CARD FRAUD DETECTION

Exploiting a NoSQL Database to detect credit card fraudulent transactions

INDICE



A. CONCEPTUAL SCHEMA **4**

Assumptions 5

Constraints 6

B. LOGICAL MODEL **7**

C. SCRIPT FOR CREATING AND LOADING **9**

I. Dataset generation (as described in the Fraud detection handbook) 9

I I. Dataset loading 12

D. SCRIPT FOR THE OPERATIONS **16**

QUERIES EXPLANATION **21**

Query A
Query B
Query C
Query D 1
Query D 2
Query E

E. PERFORMANCE ANALISYS **25**

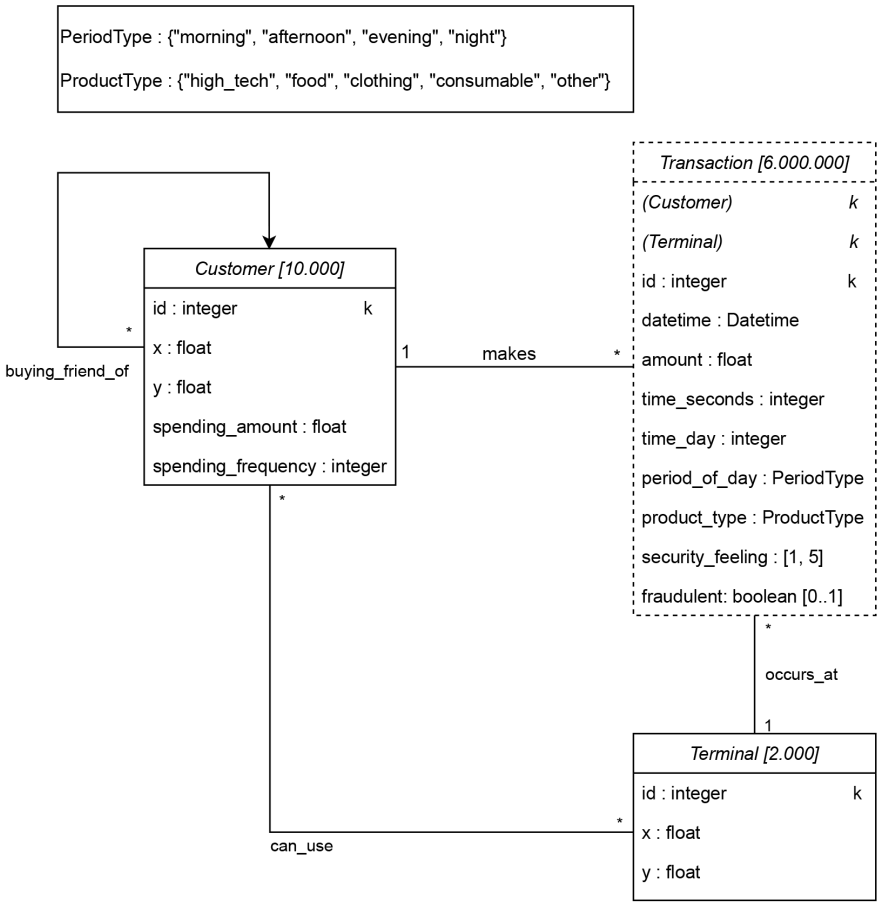
APPLE M2 PRO 10-CORE

- Dataset generation
- Dataset loading
- Queries

INTEL® CORE™ I7-9750H × 12

- Dataset generation
- Dataset loading
- Queries

A. CONCEPTUAL SCHEMA



LThe conceptual schema identifies three classes: Customer, Transaction and Terminal.
They are connected through the associations that shows the transactions executed by a customer and from which terminal they are carried out. They also indicate what are the terminals that a customer can use and which customers are related to each other as “buying friends”.

The class Transaction is a weak class and it is identified by the id of the customer that has executed it, by the id of the terminal from which it is carried out and by the time (in seconds) passed from the first transaction (first day).

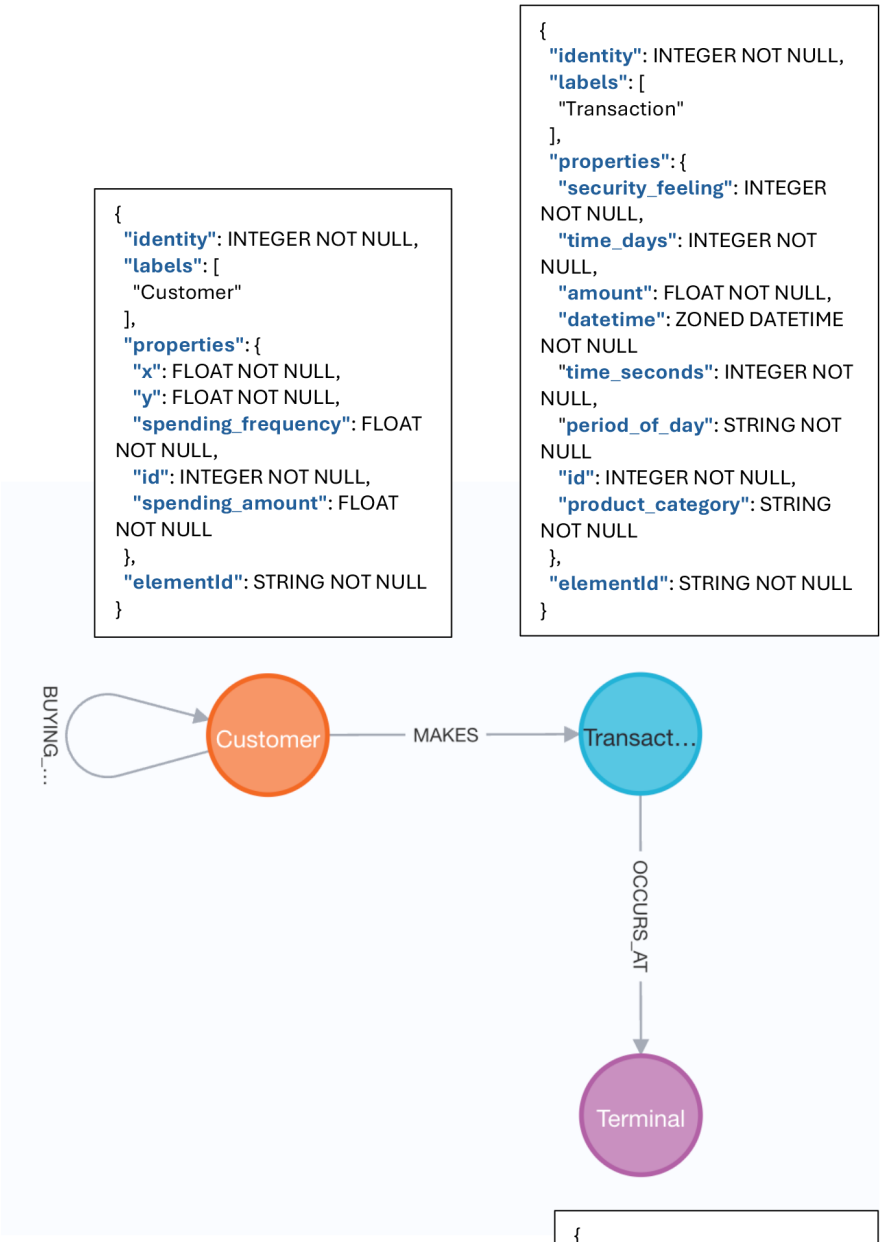
Assumptions

1. The geographic location is represented by a couple of coordinates which represent the position of a customer (or a terminal) in a two-dimensional grid 100 x 100, expressed through the two attributes x and y of both classes customer and terminal
2. The spending_amount is represented by the mean, which is a random float value generated in a range [0, 100] that represent the mean amount of money spent by a customer per day
3. The spending_frequency is represented by the mean_nb_tx_per_day, which is a random integer value generated in a range [0, 4] that represent the mean number of transactions carried out by a customer per day
4. The time_seconds attribute represents the seconds passed since the first transaction registered by the user
5. The time_day attribute represents the number of the day in which the effective transaction is carried out, which is calculated in terms of the number of days passed since the first day of transactions registered by the customer
6. The period of the day in which the transaction has been executed is a string value, taken from a list (PeriodType), and it is calculated in terms of the difference between the seconds of the transaction (time_seconds) and the days passed since the first day of transactions converted in seconds (time_day * 86400). The result is then converted in hours and converted to one of the values in the list PeriodType, by looking at the range in which it falls (for example: 6.01 – 12.00 = “morning”, 12.01: 18.01: “afternoon”…)
7. The value of the attribute product_type is chosen randomly between the values in the list ProductType
8. The value of the attribute security_feeling is generated casually from the range [1, 5] after the execution of a transaction (the fact that it is expressed by the user is just simulated)
9. The text does not provide an identifier for the terminals, so we have introduced an id to identify the different instances
10. The fraudolent attribute is a boolean value, set by the query b after having discovered the fraudulent transactions

Constraints

1. The associations between two customers that are “buying_friends” is based on the satisfaction of the following requirements: both customers must have done at least 3 transactions from the same terminal and the difference between the averages for the security_feeling attribute of those transactions, calculated on the two customers, must be lower than 1
2. The coordinates values (x and y), both for the customers and for the terminals, must be two float values between 0 and 100
3. The mean of all the amounts of transactions made by a customer must be equal to the spending_amount of the customer
4. The mean number of transactions per day made by a customer must be equal to the spending_frequency of the customer

B. LOGICAL MODEL



This logical model has been obtained reasoning on the operations required.
In particular, the operation “c”, which asks to retrieve a chain of customers and terminals, shows immediately how the choose of the Neo4J NoSQL system is the best one.
Also, the operation “a” is easier to carry out in a graph database and the operation “d2”, which asks to create the buying friend relationship can be considered as the creation of an edge from the Customer node to itself.
So, starting from the conceptual schema, we have developed this logical model.

Note: in Neo4J we have used a single id property to identify the transactions, since the nodes are identified by a unique id.

C. SCRIPT FOR CREATING AND LOADING THE DATASET

I. Dataset generation (as described in the Fraud detection handbook)

```
import datetime
import numpy as np
import pandas as pd
import pickle
from pandarallel import pandarallel
import time
import random

pandarallel.initialize(progress_bar=True)


# funzione che genera la tabella dei customers
def generate_customer_profiles_table(n_customers, random_state=0):

np.random.seed(random_state)

customer_id_properties = []

# Generate customer properties from random distributions
for customer_id in range(n_customers):

x_customer_id = np.random.uniform(0, 100)
y_customer_id = np.random.uniform(0, 100)

mean_amount = np.random.uniform(5, 100) # Arbitrary (but sensible) value
std_amount = mean_amount / 2 # Arbitrary (but sensible) value

mean_nb_tx_per_day = np.random.uniform(0, 4) # Arbitrary (but sensible) value

customer_id_properties.append(
[
customer_id,
x_customer_id,
y_customer_id,
mean_amount,
std_amount,
mean_nb_tx_per_day,
]
)

customer_profiles_table = pd.DataFrame(
customer_id_properties,
columns=[
"CUSTOMER_ID",
"x_customer_id",
"y_customer_id",
"mean_amount",
"std_amount",
"mean_nb_tx_per_day",
],
)
```

```
return customer_profiles_table


# funzione che genera la tabella dei terminals
def generate_terminal_profiles_table(n_terminals, random_state=0):

np.random.seed(random_state)

terminal_id_properties = []

# Generate terminal properties from random distributions
for terminal_id in range(n_terminals):

x_terminal_id = np.random.uniform(0, 100)
y_terminal_id = np.random.uniform(0, 100)

terminal_id_properties.append([terminal_id, x_terminal_id, y_terminal_id])

terminal_profiles_table = pd.DataFrame(
terminal_id_properties,
columns=["TERMINAL_ID", "x_terminal_id", "y_terminal_id"],
)

return terminal_profiles_table


# funzione che associa i customers con i terminals
def get_list_terminals_within_radius(customer_profile, x_y_terminals, r):

# Use numpy arrays in the following to speed up computations

# Location (x,y) of customer as numpy array
x_y_customer = customer_profile[["x_customer_id", "y_customer_id"]].values.astype(
float
)

# Squared difference in coordinates between customer and terminal locations
squared_diff_x_y = np.square(x_y_customer - x_y_terminals)

# Sum along rows and compute suared root to get distance
dist_x_y = np.sqrt(np.sum(squared_diff_x_y, axis=1))

# Get the indices of terminals which are at a distance less than r
available_terminals = list(np.where(dist_x_y < r)[0])

# Return the list of terminal IDs
return available_terminals


# funzione che genera le transactions
def generate_transactions_table(customer_profile, start_date, nb_days=10):

customer_transactions = []

random.seed(int(customer_profile.CUSTOMER_ID))
np.random.seed(int(customer_profile.CUSTOMER_ID))

# For all days
for day in range(nb_days):

# Random number of transactions for that day
nb_tx = np.random.poisson(customer_profile.mean_nb_tx_per_day)
```

```
# If nb_tx positive, let us generate transactions
if nb_tx > 0:

for tx in range(nb_tx):

# Time of transaction: Around noon, std 20000 seconds. This choice aims at simulating the fact
that
# most transactions occur during the day.
time_tx = int(np.random.normal(86400 / 2, 20000))

# If transaction time between 0 and 86400, let us keep it, otherwise, let us discard it
if (time_tx > 0) and (time_tx < 86400):

# Amount is drawn from a normal distribution
amount = np.random.normal(
customer_profile.mean_amount, customer_profile.std_amount
)

# If amount negative, draw from a uniform distribution
if amount < 0:
amount = np.random.uniform(0, customer_profile.mean_amount * 2)

amount = np.round(amount, decimals=2)

if len(customer_profile.available_terminals) > 0:

terminal_id = random.choice(
customer_profile.available_terminals
)

customer_transactions.append(
[
time_tx + day * 86400,
day,
customer_profile.CUSTOMER_ID,
terminal_id,
amount,
]
)

customer_transactions = pd.DataFrame(
customer_transactions,
columns=[
"TX_TIME_SECONDS",
"TX_TIME_DAYS",
"CUSTOMER_ID",
"TERMINAL_ID",
"TX_AMOUNT",
],
)

if len(customer_transactions) > 0:
customer_transactions["TX_DATETIME"] = pd.to_datetime(
customer_transactions["TX_TIME_SECONDS"], unit="s", origin=start_date
)
customer_transactions = customer_transactions[
[
"TX_DATETIME",
"CUSTOMER_ID",
"TERMINAL_ID",
"TX_AMOUNT",
"TX_TIME_SECONDS",
"TX_TIME_DAYS",
]
]
```

```
return customer_transactions


days = [120, 265, 537]


# funzione che genera il dataset
def generate_dataset(dataset_selection):
index = int(dataset_selection)
n_customers = 5000
n_terminals = 10000
nb_days = days[index - 1]

start_date = "2025-01-01 00:00:00"
r = 5
start_time = time.time()
customer_profiles_table = generate_customer_profiles_table(
n_customers, random_state=0
)
print(
"Time to generate customer profiles table: {0:.2}s".format(
time.time() - start_time
)
)

start_time = time.time()
terminal_profiles_table = generate_terminal_profiles_table(
n_terminals, random_state=1
)
print(
"Time to generate terminal profiles table: {0:.2f}s".format(
time.time() - start_time
)
)

start_time = time.time()
x_y_terminals = terminal_profiles_table[
["x_terminal_id", "y_terminal_id"]
].values.astype(float)

# With Pandarallel
def func(cust, x_y_terminals=x_y_terminals):
return get_list_terminals_within_radius(cust, x_y_terminals=x_y_terminals, r=r)

customer_profiles_table["available_terminals"] = (
customer_profiles_table.parallel_apply(
func,
axis=1,
)
)
customer_profiles_table["nb_terminals"] = (
customer_profiles_table.available_terminals.apply(len)
)
print()
print(
"Time to associate terminals to customers: {0:.2f}s".format(
time.time() - start_time
)
)
```

```
start_time = time.time()

# With Pandarallel
def func2(cust, nb_days=nb_days):
return generate_transactions_table(cust.iloc[0], start_date, nb_days=nb_days)

transactions_df = (
customer_profiles_table.groupby("CUSTOMER_ID")
.parallel_apply(func2)
.reset_index(drop=True)
)
print()
print("Time to generate transactions: {0:.2f}s".format(time.time() - start_time))

# Sort transactions chronologically
transactions_df = transactions_df.sort_values("TX_DATETIME")
# Reset indices, starting from 0
transactions_df.reset_index(inplace=True, drop=True)
transactions_df.reset_index(inplace=True)
# TRANSACTION_ID are the dataframe indices, starting from 0
transactions_df.rename(columns={"index": "TRANSACTION_ID"}, inplace=True)

with open(
"dataset/dataset " + str(dataset_selection) + "/customers.pkl", "wb"
) as f:
pickle.dump(customer_profiles_table, f)

with open(
"dataset/dataset " + str(dataset_selection) + "/terminals.pkl", "wb"
) as f:
pickle.dump(terminal_profiles_table, f)

with open(
"dataset/dataset " + str(dataset_selection) + "/transactions.pkl", "wb"
) as f:
pickle.dump(transactions_df, f)

 from os import stat

 print(
 f"Peso dataset: { (stat("dataset/dataset "+dataset_selection+"/customers.pkl").st_size +
 stat("dataset/dataset "+dataset_selection+"/terminals.pkl").st_size + stat("dataset/dataset
 "+dataset_selection+"/transactions.pkl").st_size)/1000000}"
 )
```

I I. Dataset loading

The script import_dataset for loading the dataset contains different functions that reads the datasete previously generated and send the queries to create the nodes and import the data into the Neo4J db instance through the Python driver for Neo4J.

For each dataset there are two type of functions. One that calls the Python driver function to send the query to the db through the Python driver and the other one that implements the query through a batch method, to improve the performances and reduce the amount of memory asked to the machine.

The functions insert_customers, insert_terminals and insert_transactions are of the first type of function described above while the load_customers_in_batch, load_terminals_in_batch and load_transactions_in_batch are of the second type.

```
import logging
import time
import pickle
from neo4j import GraphDatabase
from concurrent.futures import ThreadPoolExecutor

URI = "bolt://localhost:7687"
AUTH = ("neo4j", "provaprova")
driver = GraphDatabase.driver(URI, auth=AUTH)

logging.basicConfig(
level=logging.INFO, format="%(asctime)s - %(levelname)s - %(message)s"
)


def create_indexes():
with driver.session() as session:
session.run("CREATE INDEX IF NOT EXISTS FOR (c:Customer) ON (c.id);")
session.run("CREATE INDEX IF NOT EXISTS FOR (t:Terminal) ON (t.id);")
session.run("CREATE INDEX IF NOT EXISTS FOR (tx:Transaction) ON (tx.id);")
logging.info("Indexes created successfully.")


def insert_dataset(dataset_selection):
create_indexes()
insert_terminals(dataset_selection)
insert_customers(dataset_selection)
insert_transactions(dataset_selection)
```

```
def load_customers_in_batch(tx, data, batch_size=5000):
query = """
UNWIND $customers AS customer
MERGE (c:Customer {id: customer.customer_id})
ON CREATE SET
c.x = customer.x_customer_id,
c.y = customer.y_customer_id,
c.spending_amount = customer.spending_amount,
c.spending_frequency = customer.spending_frequency
WITH c, customer.available_terminals AS terminals
UNWIND terminals AS terminal_id
MATCH (t:Terminal {id: terminal_id})
CREATE (c)-[:CAN_USE]->(t);
"""

for i in range(0, len(data), batch_size):
batch = data[i : i + batch_size]
customers = [
{
"customer_id": row["CUSTOMER_ID"],
"x_customer_id": row["x_customer_id"],
"y_customer_id": row["y_customer_id"],
"spending_amount": row["mean_amount"],
"spending_frequency": row["mean_nb_tx_per_day"],
"available_terminals": row["available_terminals"],
}
for row in batch
]
try:
logging.info(f"Processing customers batch {i} to {i + batch_size}")
tx.run(query, customers=customers)
except Exception as e:
logging.error(f"Error in batch {i}-{i + batch_size}: {e}")


def insert_customers(dataset_selection):
start_time = time.time()
try:
with open(f"dataset/dataset {dataset_selection}/customers.pkl", "rb") as f:
customers = pickle.load(f)
customer_profiles_list = customers.to_dict(orient="records")

with driver.session() as session:
session.execute_write(load_customers_in_batch, customer_profiles_list)

logging.info(f"Customers imported in {time.time() - start_time:.2f} s")
except Exception as e:
logging.error(f"Error during customer import: {e}")


def load_terminals_in_batch(tx, data, batch_size=5000):
query = """
UNWIND $terminals AS terminal
MERGE (t:Terminal {id: terminal.terminal_id})
ON CREATE SET
t.x = terminal.x_terminal_id,
t.y = terminal.y_terminal_id
"""
```

```
for i in range(0, len(data), batch_size):
batch = data[i : i + batch_size]
terminals = [
{
"terminal_id": row["TERMINAL_ID"],
"x_terminal_id": row["x_terminal_id"],
"y_terminal_id": row["y_terminal_id"],
}
for row in batch
]

try:
logging.info(f"Processing terminals batch {i} to {i + batch_size}")
tx.run(query, terminals=terminals)
except Exception as e:
logging.error(f"Error in batch {i}-{i + batch_size}: {e}")


def insert_terminals(dataset_selection):
start_time = time.time()
try:
with open(f"dataset/dataset {dataset_selection}/terminals.pkl", "rb") as f:
terminals = pickle.load(f)
terminal_profiles_list = terminals.to_dict(orient="records")

with driver.session() as session:
session.execute_write(load_terminals_in_batch, terminal_profiles_list)

logging.info(f"Terminals imported in {time.time() - start_time:.2f} s")
except Exception as e:
logging.error(f"Error during terminal import: {e}")


def load_transactions_in_batch(tx, transactions):
query = """
UNWIND $transactions AS transaction
CREATE (t:Transaction {
id: transaction.transaction_id,
datetime: datetime(transaction.tx_datetime),
amount: transaction.tx_amount,
time_seconds: transaction.tx_time_seconds,
time_days: transaction.tx_time_days
})
WITH t, transaction
MATCH (c:Customer {id: transaction.customer_id})
MATCH (term:Terminal {id: transaction.terminal_id})
CREATE (c)-[:MAKES]->(t)
CREATE (t)-[:OCCURS_AT]->(term);
"""

try:
tx.run(query, transactions=transactions)
except Exception as e:
logging.error(f"Error during batch insertion: {e}")

 def insert_transactions(dataset_selection, batch_size=5000):
 start_time = time.time()
 try:
 with open(f"dataset/dataset {dataset_selection}/transactions.pkl", "rb") as f:
 transactions = pickle.load(f)
 transactions_list = transactions.to_dict(orient="records")
```

```
with driver.session() as session:
for i in range(0, len(transactions_list), batch_size):
batch = transactions_list[i : i + batch_size]
transactions_batch = [
{
"transaction_id": row["TRANSACTION_ID"],
"tx_datetime": row["TX_DATETIME"],
"tx_amount": row["TX_AMOUNT"],
"tx_time_seconds": row["TX_TIME_SECONDS"],
"tx_time_days": row["TX_TIME_DAYS"],
"customer_id": row["CUSTOMER_ID"],
"terminal_id": row["TERMINAL_ID"],
}
for row in batch
]

logging.info(f"Processing transactions batch {i} - {i + batch_size}...")
session.execute_write(load_transactions_in_batch, transactions_batch)

logging.info(f"Transactions imported in {time.time() - start_time:.2f} s")
except Exception as e:
logging.error(f"Error during transaction import: {e}")
driver.close()
```

D. SCRIPT FOR THE OPERATIONS

```
import csv
import logging
import time
from neo4j import GraphDatabase

URI = "bolt://localhost:7687"
AUTH = ("neo4j", "provaprova")
driver = GraphDatabase.driver(URI, auth=AUTH)

# Set up logging
logging.basicConfig(
level=logging.INFO, format="%(asctime)s - %(levelname)s - %(message)s"
)


def query_a():
# explanation of the query in the report
query = """
MATCH (c1:Customer)-[:MAKES]->(t1:Transaction)-[:OCCURS_AT]->(term:Terminal)
MATCH (c2:Customer)-[:MAKES]->(t2:Transaction)-[:OCCURS_AT]->(term)
WHERE c1.id < c2.id
WITH c1, c2, COUNT(DISTINCT term) AS shared_terminals,
avg(t1.amount) AS avg_spent_c1, avg(t2.amount) AS avg_spent_c2
WHERE shared_terminals >= 3
AND ABS(avg_spent_c1 - avg_spent_c2) / avg_spent_c1 < 0.1
RETURN c1.id, avg_spent_c1, c2.id, avg_spent_c2, shared_terminals
ORDER BY c1.id
"""
start_time = time.time()

try:
logging.info("Processing query A")
# execution of the query on the database through the usage of a driver session
with driver.session() as session:
result = session.run(query)
data = result.data()

logging.info(f"Query A executed successfully")
file_path = "query_results/A.csv"

# writing query results in a .csv file
with open(file_path, mode="w", newline="") as file:
writer = csv.writer(file)
writer.writerow(
[
"Customer_c1",
"Avg_spent_c1",
"Customer_c2",
"Avg_spent_c2",
"Shared_terminals",
]
)
for row in data:
writer.writerow(
[
```

```
[
row["c1.id"],
f"{row['avg_spent_c1']:.2f}",
row["c2.id"],
f"{row['avg_spent_c2']:.2f}",
row["shared_terminals"],
]
)

logging.info("Results written to A.csv")

except Exception as e:
logging.error(f'Error "{e}" while processing query A')

execution_time = time.time() - start_time
logging.info(f"Execution time for query A: {execution_time:.2f} s")
driver.close()


def query_b():
# explanation of the query in the report
query = """
MATCH (t:Transaction)-[:OCCURS_AT]->(term:Terminal)
WHERE ((t.datetime.year = date().year AND t.datetime.month = date().month - 1)
OR (date().month = 1 AND t.datetime.month = 12 AND t.datetime.year = date().year -1))
WITH term.id AS term_id, AVG(t.amount) AS prev_month_avg
MATCH (t2:Transaction)-[:OCCURS_AT]->(term:Terminal)
WHERE term.id = term_id AND (t2.datetime.year = date().year AND t2.datetime.month = date().month)
AND t2.amount > 1.2*prev_month_avg
SET t2.fraudulent = TRUE
"""

start_time = time.time()

try:
logging.info("Processing query B")
# execution of the query on the database through the usage of a driver session
with driver.session() as session:
result = session.run(query)
data = result.data()

logging.info(f"Query B executed successfully")

except Exception as e:
logging.error(f'Error "{e}" while processing query B')

execution_time = time.time() - start_time
logging.info(f"Execution time for query B: {execution_time:.2f} s")
driver.close()


def query_c(customer_id=0, k=3):
# explanation of the query in the report
start_time = time.time()
try:

logging.info("Processing query C")
co_customers = set()

# execution of the query on the database through the usage of a driver session
with driver.session() as session:
current_level_customers = {customer_id}
```

```
for _ in range(k):
query = """
MATCH (u:Customer)-[:MAKES]->(t:Transaction)-[:OCCURS_AT]->(term:Terminal)
<-[:OCCURS_AT]-(t2:Transaction)<-[:MAKES]-(u2:Customer)
WHERE u.id IN $current_customers AND u2.id <> u.id AND u2.id <> $u_id
RETURN DISTINCT u2.id AS co_customer_id
"""
result = session.run(
query,
u_id=customer_id,
current_customers=list(current_level_customers),
)

next_level_customers = {record["co_customer_id"] for record in result}
next_level_customers -= co_customers

if not next_level_customers:
break

co_customers.update(next_level_customers)
current_level_customers = next_level_customers

file_path = "query_results/C.csv"

# writing query results in a .csv file
with open(file_path, mode="w", newline="") as file:
writer = csv.writer(file)
writer.writerow(["Customer_id", "Cocustomer_ids"])
writer.writerow([customer_id, list(co_customers)[0]])
for i in range(1, len(co_customers)):
writer.writerow(["", list(co_customers)[i]])

logging.info("Results written to C.csv")

except Exception as e:
logging.error(f'Error "{e}" while processing query C')

execution_time = time.time() - start_time
logging.info(f"Execution time for query C: {execution_time:.2f} s")
driver.close()


def query_d():
# explanation of the query in the report
query = """
MATCH (t:Transaction)
WITH t, datetime(t.datetime).hour AS hour
SET
t.period_of_day = CASE
WHEN hour >= 6 AND hour < 12 THEN 'morning'
WHEN hour >= 12 AND hour < 18 THEN 'afternoon'
WHEN hour >= 18 AND hour < 24 THEN 'evening'
ELSE 'night'
END,
t.product_category = ['high-tech', 'food', 'clothing', 'consumable', 'other'][toInteger(rand() *
5)],
t.security_feeling = toInteger(rand() * 5) + 1;
"""

start_time = time.time()
```

```
try:
logging.info("Processing query D")
# execution of the query on the database through the usage of a driver session
with driver.session() as session:
session.run(query)
logging.info(f"Query D executed successfully")
except Exception as e:
logging.error(f'Error "{e}" while processing query D')

execution_time = time.time() - start_time
logging.info(f"Execution time for query D: {execution_time:.2f} s")
driver.close()


def query_d_2_batch(tx, data, batch_size=5000):
# explanation of the query in the report
query = """
UNWIND $customers AS pair
MATCH (c1:Customer {id: pair.c1_id})
MATCH (c2:Customer {id: pair.c2_id})
CREATE (c1)-[:BUYING_FRIEND_OF]->(c2)
"""

for i in range(0, len(data), batch_size):
batch = data[i : i + batch_size]
customers = [{"c1_id": row["c1_id"], "c2_id": row["c2_id"]} for row in batch]
try:
logging.info(
f"Creating buying friends relationship batch {i} to {i + batch_size}"
)
tx.run(query, customers=customers)
except Exception as e:
logging.error(
f"Error while processing buying friends relationship batch {i}-{i + batch_size}: {e}"
)


def query_d_2():
# explanation of the query in the report
query = """
MATCH (c1:Customer)-[:MAKES]->(t1:Transaction)-[:OCCURS_AT]->(term:Terminal)
MATCH (c2:Customer)-[:MAKES]->(t2:Transaction)-[:OCCURS_AT]->(term)
WHERE c1.id < c2.id
WITH c1, c2, COUNT(DISTINCT t1) AS num_transactions_c1,COUNT(DISTINCT t2) AS num_transactions_c2,
avg(t1.security_feeling) AS avg_security_feeling_c1, avg(t2.security_feeling) AS
avg_security_feeling_c2

WHERE num_transactions_c1 > 3 AND num_transactions_c2 > 3
AND ABS(avg_security_feeling_c1 - avg_security_feeling_c2) < 1
RETURN c1.id as c1_id, c2.id as c2_id
"""
start_time = time.time()

try:
logging.info("Processing query D 2")
# execution of the query on the database through the usage of a driver session
with driver.session() as session:
result = session.run(query)
data = result.data()

data = list(data)
session.execute_write(query_d_2_batch, data)
```

```
logging.info(f"Time to create relationships: {time.time() - start_time:.2f} s")
except Exception as e:
logging.error(f"Error during relationships creation: {e}")


def query_e():
# explanation of the query in the report
query = """
MATCH (t:Transaction)
WITH t.period_of_day AS period,
COUNT(t) AS num_transactions_per_period,
COUNT(CASE WHEN t.fraudulent = TRUE THEN 1 ELSE NULL END) AS num_fraud_transactions
RETURN period,
num_transactions_per_period,
num_fraud_transactions,
num_fraud_transactions * 100.0 / num_transactions_per_period AS fraud_rate
"""

start_time = time.time()

try:
logging.info("Processing query E")
# execution of the query on the database through the usage of a driver session
with driver.session() as session:
result = session.run(query)
data = result.data()

logging.info("Query E executed successfully")

# writing query results in a .csv file
file_path = "query_results/E.csv"
with open(file_path, mode="w", newline="") as file:
writer = csv.writer(file)
writer.writerow(
[
"Period",
"Num_Transactions",
"Num_Fraud_Transactions",
"Fraud_Rate(%)",
]
)
  for row in data:
  writer.writerow(
  [
  row["period"],
  row["num_transactions_per_period"],
  row["num_fraud_transactions"],
  f"{row['fraud_rate']:.4f}",
  ]
  )

  logging.info(f"Results written to {file_path}")

  except Exception as e:
  logging.error(f'Error "{e}" while processing query E')

  execution_time = time.time() - start_time
  logging.info(f"Execution time for query E: {execution_time:.2f} s")
  driver.close()
```

Queries explanation

Query A

```
MATCH (c1:Customer)-[:MAKES]->(t1:Transaction)-[:OCCURS_AT]->(term:Terminal)
MATCH (c2:Customer)-[:MAKES]->(t2:Transaction)-[:OCCURS_AT]->(term)
WHERE c1.id < c2.id
WITH c1, c2, COUNT(DISTINCT term) AS shared_terminals,
avg(t1.amount) AS avg_spent_c1, avg(t2.amount) AS avg_spent_c2
WHERE shared_terminals >= 3
AND ABS(avg_spent_c1 - avg_spent_c2) / avg_spent_c1 < 0.1
RETURN c1.id, avg_spent_c1, c2.id, avg_spent_c2, shared_terminals
ORDER BY c1.id
```

We use two MATCH to take all the customer pairs that shares terminal where they made at least one transaction, the WHERE clause avoid to pick wrong pairs (ex. <Y,X> when we already have <X,Y> and <X,X>); c1 and c2 are needed for the ids; we assumed that the spending amount considered was the average of the single transaction amount, so we used the avg() function to create avg_spent_c1 and avg_spent_c2; then we count how many distinct terminals <X,Y> shares (shared_terminals). In the last WHERE clause we select only the couple having at least 3 terminals in common and respects the property (difference between the spending amount < 10%). We return c1.id, avg_spent_c1, c2.id, avg_spent_c2, shared_terminals, ordered by c1.id to facilitate the query validation.

Query B

```
MATCH (t:Transaction)-[:OCCURS_AT]->(term:Terminal)
WHERE ((t.datetime.year = date().year AND t.datetime.month = date().month - 1)
OR (date().month = 1 AND t.datetime.month = 12 AND t.datetime.year = date().year -1))
WITH term.id AS term_id, AVG(t.amount) AS prev_month_avg
MATCH (t2:Transaction)-[:OCCURS_AT]->(term:Terminal)
WHERE term.id = term_id AND (t2.datetime.year = date().year AND t2.datetime.month = date().month)
AND t2.amount > 1.2*prev_month_avg
SET t2.fraudulent = TRUE
```

We take all the transactions made in the previous month with the first MATCH AND WHERE, the clause handles the possibility that the previous month is in a different year (December or January) ; WITH takes the terminal id and the average of the transaction amount made on it the previous month.
We take the transactions made in the current month, thanks to the second MATCH AND WHERE, in which the second part of the clause checks the property requested for identifying the possible fraud transactions. Finally, we add a boolean property to the fraud transaction that will be used by query E.

Query C

```
co_customers = set()
current_level_customers = {customer_id}
for _ in range(k):
                query = """
                      MATCH (u:Customer)-[:MAKES]->(t:Transaction)-[:OCCURS_AT]-
>(term:Terminal)<-[:OCCURS_AT]-(t2:Transaction)<-[:MAKES]-(u2:Customer)
                WHERE u.id IN $current_customers AND u2.id <> u.id AND u2.id <> $u_id
                RETURN DISTINCT u2.id AS co_customer_id
                """
                result = session.run(
                    query,
                    u_id=customer_id,
                    current_customers=list(current_level_customers),
                )

                next_level_customers = {record["co_customer_id"] for record in result}
                next_level_customers -= co_customers

                if not next_level_customers:
                    break

                co_customers.update(next_level_customers)
                current_level_customers = next_level_customers
```

First of all, we initialize an empty set (co_customers) to store the unique co-customers found.
Then, given a customer_id and initializing current_level_customers with this ID, we start to iterate from 0 to k to find co-customers at increasing degrees.
In each iteration a query is executed to find the u2 (customers) who made transactions at the same terminal as customers in current_level_customers (u) (MATCH).

The WHERE clause ensures that the u.id is IN $current_customers, so it only expands from the current set of customers, then it checks that u2.id <> u.id, because a customer cannot be their own co-customer and finally that u2.id <> $u_id, which ensures to eliminate the original customer_id from the results.
The query returns distinct co-customer IDs (u2.id).

Then we update the co_customer set, the newly found co_customer_ids are stored in next_level_customers.
The expression next_level_customers -= co_customers ensures that previously discovered customers are not reprocessed, to avoid duplicates.
If no new co-customers are found, the loop breaks early, otherwise co_customers is updated and current_level_customers moves to the next level.

The loop terminates when:

- the maximum degree k is reached, so we have navigated the chain we were interested in
- no new co-customers are found at the current level

Finally, the result is a set of co-customers of the given customer up to degree k.

Query D 1

```
MATCH (t:Transaction)
WITH t, datetime(t.datetime).hour AS hour
SET
t.period_of_day = CASE
WHEN hour >= 6 AND hour < 12 THEN 'morning'
WHEN hour >= 12 AND hour < 18 THEN 'afternoon'
WHEN hour >= 18 AND hour < 24 THEN 'evening'
ELSE 'night'
END,
t.product_category = ['high-tech', 'food', 'clothing', 'consumable', 'other'][toInteger(rand() *
5)],
t.security_feeling = toInteger(rand() * 5) + 1;
```

We take all the transactions with MATCH; we add the new properties with SET: period_of_day depending on the t.datetime value (hour), product_category and security feeling randomly.

Query D 2

```
MATCH (c1:Customer)-[:MAKES]->(t1:Transaction)-[:OCCURS_AT]->(term:Terminal)
MATCH (c2:Customer)-[:MAKES]->(t2:Transaction)-[:OCCURS_AT]->(term)
WHERE c1.id < c2.id
WITH c1, c2, COUNT(DISTINCT t1) AS num_transactions_c1,COUNT(DISTINCT t2) AS num_transactions_c2,
avg(t1.security_feeling) AS avg_security_feeling_c1, avg(t2.security_feeling) AS
avg_security_feeling_c2
WHERE num_transactions_c1 > 3 AND num_transactions_c2 > 3 AND ABS(avg_security_feeling_c1 -
avg_security_feeling_c2) < 1
RETURN c1.id as c1_id, c2.id as c2_id

UNWIND $customers AS pair
MATCH (c1:Customer {id: pair.c1_id})
MATCH (c2:Customer {id: pair.c2_id})
CREATE (c1)-[:BUYING_FRIEND_OF]->(c2)
```

The first part calculates the relationships between customers based on certain conditions, and the second part creates relationships between those customers. The first MATCH
finds all customers who have made a transaction that occurred at a terminal (term);

The second MATCH finds another customer who also made a transaction at the same terminal. The WHERE clause avoid to take duplicates. The WITH aggregates the data, calculating:

- number of transactions made by the first customer
- number of transactions made by the second customer
- the average of security feeling of the first customer transactions
- the average of security feeling of the second customer transactions

The last WHERE clause filters the pairs of customers that have the property requested

The return part prepares the pairs of customers that will be the input data for the second part of the query that create the relationship “buying_friend_of” between them.

Query E

```
MATCH (t:Transaction)
WITH t.period_of_day AS period, COUNT(t) AS num_transactions_per_period, COUNT(CASE WHEN
t.fraudulent = TRUE THEN 1 ELSE NULL END) AS num_fraud_transactions
RETURN period, num_transactions_per_period,num_fraud_transactions, num_fraud_transactions * 100.0
/ num_transactions_per_period AS fraud_rate
```

We take all the transactions, the WITH separates the transactions with the “period_of_day” property, counting how many transactions belong to every period and how many of those are fraudulent using the “fraudulent” property (set in query b). Finally, we calculate the fraud rate (percentage of fraudulent transactions) for each period.

E. PERFORMANCES

50MB DATASET =	5,000 customers,	10,000 terminals,	1,150,370 transactions
100MB DATASET =	5,000 customers,	10,000 terminals,	2,540,013 transactions
200MB DATASET =	5,000 customers,	10,000 terminals,	5,149,084 transactions

Apple M2 Pro 10-core

Dataset generation

	Terminals	Customers	Transactions
50MB	0.03 s	0.047 s	3.31 s
100MB	0.03 s	0.043 s	6.11 s
200MB	0.03 s	0.042 s	11.02 s

Dataset loading

	Terminals	Customers	Transactions
50MB	0.24 s	3.03 s	64.34 s
100MB	0.23 s	3.09 s	145.83 s
200MB	0.27 s	2.98 s	279.58 s

Queries

	QUERY A	QUERY B	QUERY C	QUERY D 1	QUERY D 2	QUERY E
50MB	37.39 s	4.31 s	7.00 s	5.05 s	47.48 s	0.44 s
100MB	166.26 s	7.64 s	28.46 s	13.94 s	189.58 s	0.78 s
200MB	506.53 s	8.17 s	103.59 s	19.44 s	577.98 s	1.45 s

Intel® Core™ i7-9750H × 12

Dataset generation

	Terminals	Customers	Transactions
50MB	0.052 s	0.05 s	0.052 s
100MB	0.052 s	0.05 s	0.05 s
200MB	0.052 s	0.05 s	37.83 s

Dataset loading

	Terminals	Customers	Transactions
50MB	1.39 s	6.11 s	97.35 s
100MB	1.53 s	5.77 s	222.90 s
200MB	1.07 s	5.51 s	381.44 s

Queries

	QUERY A	QUERY B	QUERY C	QUERY D1	QUERY D2	QUERY E
50MB	47.93 s	5.12 s	12.32 s	6.73 s	66.71 s	0.83 s
100MB	206.24 s	8.23 s	42.17 s	12.67 s	237.12 s	1.29 s
200MB	789.17 s	12.49 s	150.42 s	25.21 s	611,12 s	2.94 s